

Spring Boot Cheat Sheet (Final Exam Version)

Core Annotations

@RestController: REST API controller (returns JSON)

@RequestMapping: Base URL mapping

@GetMapping / **@PostMapping** / **@PutMapping** / **@DeleteMapping**: HTTP methods

@RequestBody: JSON → Java object

@PathVariable: Value from URL path

@RequestParam: Query parameter

@Autowired & Dependency Injection

@Autowired: Automatically injects dependencies (no 'new')

```
@Autowired
private UserService userService;
```

Preferred: Constructor Injection (no @Autowired needed)

```
public UserController(UserService userService) {
    this.userService = userService;
}
```

Rule: Never use 'new' for Service or Repository

Controller Example

```
@RestController
@RequestMapping("/users")
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping
    public List<User> getAllUsers() {
        return userService.getAllUsers();
    }

    @PostMapping
    public User addUser(@RequestBody User user) {
        return userService.addUser(user);
    }
}
```

Service Layer

```
@Service
public class UserService {

    private final UserRepository userRepository;
```

```

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public User addUser(User user) {
        if (userRepository.userExists(user.getEmail())) {
            throw new RuntimeException("User already exists");
        }
        return userRepository.save(user);
    }
}

```

All validation & logic goes here

Repository

```

@Repository
public class UserRepository {

    private List<User> users = new ArrayList<>();

    public User save(User user) {
        users.add(user);
        return user;
    }

    public boolean userExists(String email) {
        return users.stream()
            .anyMatch(u -> u.getEmail().equals(email));
    }
}

```

PUT & DELETE

```

@PutMapping("/{email}")
public User updateUser(@PathVariable String email,
    @RequestBody User user)

@DeleteMapping("/{email}")
public void deleteUser(@PathVariable String email)

```

Exception Handling

```

if (!userRepository.userExists(email)) {
    throw new RuntimeException("User not found");
}

```

Unit Testing

```

@Test
public void givenUser_whenAdd_thenSuccess() {
    User user = new User("test@mail.com");

    userService.addUser(user);

    assertEquals(1, userRepository.findAll().size());
}

```

Postman Example

```
POST http://localhost:8080/users
```

```
{  
  "email": "test@mail.com",  
  "name": "John"  
}
```

Flow

Postman → Controller → Service → Repository → back → JSON

Rules

- Controller = HTTP - Service = logic + validation - Repository = data - Use dependency injection -
Test success + failure